Virtual Log Files (VLFs) in SQL Server — Wiki

1. Introduction

The transaction log is the most operationally sensitive file in any SQL Server database. It is the structure that guarantees durability, enables rollback, drives crash recovery, and feeds high-availability features such as Always On Availability Groups, transactional replication, log shipping, mirroring, and Change Data Capture. The log is implemented internally as a chain of smaller logical units called Virtual Log Files (VLFs). When VLFs are healthy you never think about them. When they are not, you see the symptoms in the form of slow recovery, slow backups, slow failovers, runaway log growth, and disk-full incidents.

This wiki covers what VLFs are, what controls them, how they fail, how to diagnose them, and how to fix them.

2. What a VLF Is

A SQL Server transaction log file (`.ldf`) is not a single contiguous structure to the engine. Internally, the file is divided into a circular sequence of Virtual Log Files. Each VLF is a chunk of the physical log used as a unit of allocation, reuse, and truncation. Log records are written sequentially across VLFs as transactions occur. When the active portion of the log reaches the end of the file, it wraps back to the beginning and reuses VLFs that no longer hold active log records.

A VLF is in one of two states at any moment. An **active** VLF contains log records still required for some purpose — an open transaction, a pending log backup under FULL recovery, a replication reader that has not yet consumed it, an Availability Group secondary that has not yet hardened the records, or simply records past the last checkpoint under SIMPLE recovery. An **inactive** VLF holds no required records and is eligible for reuse on the next wrap. Truncation does not shrink the file; it simply marks VLFs as reusable.

3. The Log Manager

The component inside the SQL Server storage engine responsible for VLFs is the Log Manager. It writes log records, tracks VLF boundaries and state, advances the head of the log on commits, coordinates with the checkpoint process to harden dirty pages, and coordinates with the log backup process (under FULL and BULK_LOGGED recovery) to decide when VLFs may be marked inactive. The Log Manager is also what creates new VLFs when the log file grows.

You do not interact with the Log Manager directly. There is no command to add, remove, resize, or relocate a VLF. The DBA shapes outcomes indirectly through file sizing, autogrowth settings, recovery model choice, and backup cadence.

4. How VLFs Are Created and Sized

VLFs are created at two moments: when the log file is initially created, and every time the log file grows. SQL Server decides how many VLFs to add based on the size of the growth chunk.

For SQL Server 2014 and later, the algorithm is: if the growth amount is less than one-eighth of the current log size, only **one** VLF is added; otherwise the historical formula applies. The historical formula is: a chunk under 64 MB produces 4 VLFs; a chunk between 64 MB and 1 GB produces 8 VLFs; a chunk over 1 GB produces 16 VLFs. SQL Server 2022 introduced minor refinements but the general behavior is unchanged.

This algorithm has direct consequences. A log that grows from 1 MB to 50 GB through hundreds of small autogrowth events ends up with thousands of small VLFs. A log that grows from 0 to 50 GB in a few large deliberate steps ends up with a few dozen large VLFs. Both reach the same final size but they perform very differently.

The practical sizing target most DBAs use is VLFs of roughly 512 MB each. To achieve that, grow the log in 8 GB chunks: each 8 GB chunk produces 16 VLFs of 512 MB. A 32 GB log built from four 8 GB growths therefore contains 64 VLFs of 512 MB — a good steady state.

5. The Log Reuse Lifecycle

A log record's path through the log is straightforward. It is written into the current active VLF. It remains active as long as anything still requires it. Once nothing requires it, the VLF holding it is marked inactive on the next checkpoint (SIMPLE recovery) or log backup (FULL/BULK_LOGGED recovery). Once inactive, the VLF is available to be overwritten when the log wraps.

Truncation is purely a status change. It never reduces the size of the log file. To reduce the physical file size you must explicitly shrink. Conversely, an inability to truncate does not corrupt the log; it simply forces growth, because every new record must go into a fresh VLF rather than reusing an old one.

The single most important diagnostic for log behavior is `sys.databases.log_reuse_wait_desc`. It tells you, in one column, why the log cannot currently be truncated. The common values are summarized in the reference section below.

6. Recovery Models and Their Effect

The recovery model controls when the engine considers a VLF eligible to become inactive. Under SIMPLE recovery, the checkpoint process truncates the inactive portion of the log automatically. Under FULL or BULK_LOGGED recovery, truncation only happens after a successful log backup. Until that log backup runs, every VLF behind the last backed-up record stays active and the log keeps growing.

Switching recovery model is consequential. Switching from FULL to SIMPLE breaks the log backup chain and forfeits point-in-time recovery until the next full backup is taken. Switching from SIMPLE to FULL has no immediate effect on log behavior — log backups will not actually start working until the next full or differential backup is taken to bridge the chain.

BULK_LOGGED is a specialized middle ground. It allows certain bulk operations (notably index rebuilds and `BULK INSERT`) to be minimally logged, reducing log volume for those operations, while still requiring log backups. Point-in-time recovery is restricted within bulk-logged intervals.

7. Reference: `log_reuse_wait_desc` Values

The values you will most commonly encounter, and what each means:

`NOTHING` — log can be truncated; no problem.

`CHECKPOINT` — checkpoint has not yet run since the last log activity. Usually transient.

`LOG_BACKUP` — database is in FULL or BULK_LOGGED recovery and a log backup is needed before truncation can occur. Either take a log backup, fix the failing log backup job, or switch to SIMPLE recovery if point-in-time recovery is not required.

`ACTIVE_TRANSACTION` — a long-running or orphaned transaction is holding the log tail. Identify and resolve it. Typical sources include a forgotten `BEGIN TRAN` in a developer session, a stuck ETL job, a long index rebuild, or a stalled distributed transaction.

`AVAILABILITY_REPLICA` — an Always On secondary replica has not yet hardened the records. Check secondary connectivity, redo lag, and synchronization mode.

`DATABASE_MIRRORING` — the mirroring partner has not yet acknowledged the records. Check mirroring connectivity and queue depth.

`REPLICATION` — transactional replication's log reader agent or a CDC capture job has not yet consumed the records. Check the agent jobs.

`OLDEST_PAGE` — under indirect checkpoint, the oldest dirty page has not yet been flushed. A manual `CHECKPOINT` clears it.

`LOG_SCAN` — a log scan is in progress. Usually transient.

Any other value is documented in Microsoft's `sys.databases` reference.

8. Diagnostics

The two most useful built-in views are `sys.dm_db_log_info` (one row per VLF, available in SQL Server 2016 SP2+ and 2017+) and `sys.dm_db_log_space_usage` (overall log size and utilization). The legacy `DBCC LOGINFO` command still works on older builds but has always been undocumented.

A first-pass health check across the instance:

```sql
SELECT
    d.name,
    d.recovery_model_desc,
    d.log_reuse_wait_desc,
    (SELECT COUNT(*) FROM sys.dm_db_log_info(d.database_id)) AS vlf_count
FROM sys.databases d
WHERE d.state_desc = 'ONLINE'
ORDER BY vlf_count DESC;
```

For a specific database, the active versus inactive split is informative:

```sql
SELECT vlf_active, COUNT(*) AS vlfs
FROM sys.dm_db_log_info(DB_ID())
GROUP BY vlf_active;
```

For long-running transactions:

```sql
DBCC OPENTRAN;
```

Or the richer instance-wide view that joins `sys.dm_tran_active_transactions`, `sys.dm_tran_session_transactions`, and `sys.dm_exec_sessions` to surface session, login, host, and SQL text for each open transaction.

9. What "Healthy" Looks Like

There is no single magic VLF count, but useful guidelines exist. Under a few hundred VLFs is generally fine. Several hundred is worth watching. A thousand or more is a problem worth fixing during the next maintenance window. Tens of thousands will produce visible symptoms — slow recovery on database startup, slow Always On failovers, slow log backups, slow restores, and sluggish replication or CDC.

A healthy log file is also stable in size. It grows to its working set after a workload reaches steady state and then does not grow further. If the log is still growing day over day, something is preventing truncation and the cause should be identified rather than papered over with more disk.

`log_reuse_wait_desc` should normally read `NOTHING`, occasionally `LOG_BACKUP` or `CHECKPOINT` between operations. Anything else persisting for more than a few minutes deserves investigation.

10. Common Pathologies

The recurring causes of "wild" VLFs and runaway logs in production fall into a small number of categories.

Tiny autogrowth increments. The single most common cause of VLF fragmentation. Default percentage growth (10%) or small fixed increments produce many small growth events, each adding 4 small VLFs. Logs in the wild routinely accumulate thousands of VLFs this way. The fix is to set autogrowth to a fixed size of 512 MB or 1 GB and to pre-size the log to its expected steady-state.

Long-running or orphaned transactions. A single open transaction pins the log tail, preventing any VLFs behind it from being truncated. The log grows without bound until the transaction commits, rolls back, or the session is killed. `DBCC OPENTRAN` and `sys.dm_tran_active_transactions` find them.

Missing or broken log backups under FULL recovery. A database is placed into FULL recovery (often because it joined an Always On group or because someone set it that way without thinking) but log backups are never configured, or the job silently fails. The log can never truncate. This is the most common cause of disk-full incidents on log drives.

Stalled downstream readers. Transactional replication's log reader agent, CDC capture jobs, log shipping, and Always On secondaries all hold VLFs active until they consume them. Any of them falling behind will pin the log.

Massive single transactions. A `DELETE` of millions of rows in a single statement, or an index rebuild of a very large table under FULL recovery, generates a large active log section that must persist until commit. Batching the work and choosing recovery model carefully mitigates this.

Stuck distributed transactions. Orphaned MSDTC transactions can hold the log indefinitely with no obvious culprit in normal monitoring views.

Repeated shrink–grow cycles. Routinely shrinking the log only forces it to grow again under load, fragmenting it badly each time. Shrinking should be a deliberate one-time operation, not part of a maintenance plan.

11. Remediation Patterns

The two safe, reversible, low-risk fixes that can be automated are correcting bad autogrowth settings and issuing a manual `CHECKPOINT` for `OLDEST_PAGE` waits. Both are metadata-only or near-instantaneous operations.

Everything else requires DBA judgment. Fixing a high VLF count means backing up the log, shrinking the log file to release fragmented VLFs, and growing it back deliberately in 8 GB chunks. This requires a maintenance window and a working backup. Fixing a `LOG_BACKUP` wait may mean configuring a backup job or, if point-in-time recovery is not actually needed, switching to SIMPLE recovery — a decision the DBA makes, not a script. Fixing an `ACTIVE_TRANSACTION` wait means identifying the offending session and deciding whether to wait, kill, or escalate to the application owner. Fixing replication, AG, and mirroring waits means investigating the downstream reader, not the database itself.

The full procedure for rebuilding a fragmented log to a clean 32 GB target:

```sql
USE [YourDatabase];
GO

-- 1. Take a log backup (skip under SIMPLE recovery)
BACKUP LOG [YourDatabase] TO DISK = 'X:\Backups\YourDatabase.trn'
WITH COMPRESSION, INIT;

-- 2. Shrink to minimum to release fragmented VLFs
DBCC SHRINKFILE (N'YourDatabase_log', 0);

-- 3. Grow back deliberately in 8 GB chunks
ALTER DATABASE [YourDatabase] MODIFY FILE (NAME = N'YourDatabase_log', SIZE =  8192M
ALTER DATABASE [YourDatabase] MODIFY FILE (NAME = N'YourDatabase_log', SIZE = 16384M
ALTER DATABASE [YourDatabase] MODIFY FILE (NAME = N'YourDatabase_log', SIZE = 24576M
ALTER DATABASE [YourDatabase] MODIFY FILE (NAME = N'YourDatabase_log', SIZE = 32768M

-- 4. Set sane autogrowth for future events
ALTER DATABASE [YourDatabase]
    MODIFY FILE (NAME = N'YourDatabase_log', FILEGROWTH = 8192MB);

-- 5. Verify
SELECT COUNT(*) AS vlf_count FROM sys.dm_db_log_info(DB_ID());
```

A successful rebuild produces a low VLF count (in the dozens, not thousands), uniform VLF sizes, and a stable log file that does not need to autogrow under normal load.

12. Interaction with Other Features

Always On Availability Groups. AG secondaries hold VLFs active until they harden the records. A disconnected, slow, or far-behind secondary will pin the log on the primary. AG also requires

FULL recovery, which means log backup discipline is mandatory. Failover time is sensitive to VLF count because recovery on the new primary scans the log.

Transactional replication. The log reader agent reads committed transactions out of the log and writes them to the distribution database. If it is stopped, slow, or failing, VLFs accumulate. `log_reuse_wait_desc` reports `REPLICATION`.

Change Data Capture (CDC). CDC's capture job reads the log similarly. If it is disabled, broken, or behind, VLFs accumulate and `log_reuse_wait_desc` reports `REPLICATION` (CDC reuses the replication infrastructure for this purpose).

Database mirroring. Now deprecated, but still found in older estates. The mirror must acknowledge records before they can be truncated. `log_reuse_wait_desc` reports `DATABASE_MIRRORING`.

Log shipping. A standby database is restored from log backups produced on the primary. Log shipping does not itself prevent truncation on the primary — log backup does that — but a broken log shipping copy or restore job means recovery point objectives are silently slipping.

13. Best Practices for New Databases

The cheapest moment to get the log right is at database creation. Pre-size the log based on expected workload — peak transaction rate, longest-running transaction window, time between log backups, and any planned heavy operations such as nightly index maintenance. A reasonable starting point for many OLTP databases is 8–32 GB, sized in deliberate growth steps to produce a clean VLF layout.

Set autogrowth to a fixed size, never a percentage. 1 GB is a defensible default for most workloads; 512 MB for smaller systems; 8 GB for heavy ones. Never leave it at the historical 10% default.

Choose the recovery model deliberately. If the database does not require point-in-time recovery and is not part of an AG, mirroring, or log shipping topology, SIMPLE is appropriate and removes a category of operational risk. Otherwise, FULL with reliable log backups every 15–30 minutes is the standard.

Place the log on its own dedicated volume where possible. Logs are written sequentially and benefit from low-latency, write-optimized storage that does not contend with random data-file I/O.

14. References

Microsoft Learn: "Transaction log architecture and management guide", "sys.databases (Transact-SQL)", "sys.dm_db_log_info (Transact-SQL)", "sys.dm_db_log_space_usage (Transact-SQL)", "Recovery Models (SQL Server)".

Paul Randal, SQLskills: foundational writing on VLF behavior, the SQL Server 2014 algorithm change, and the log truncation lifecycle.

Microsoft KB 2028436: discussion of VLF count impact on recovery and AG performance.

The companion documents in this kit — `01_VLF_Discovery.sql`, `02_VLF_AutoFix_And_Log.sql`, and the operational how-to — automate the diagnostic and remediation patterns described here.